第6回：意味で探す検索

Embeddingと類似度を理解する

今日のゴールと流れ

文章のベクトル化と、検索の比較を体験する。

時間	内容
10分	キーワード検索の失敗の復習
20分	Embeddingと類似度
20分	登録処理と質問時の検索
30分	計算の穴埋め・検索の比較
10分	更新・費用・提出

言い方が違う質問

質問：「ページを閉じてもデータが残るのはなぜ？」
資料：「SQLiteで永続化する」

同じ単語がなくても、関連する文章として探したい場面があります。
Embeddingモデルで文章を数値の列へ変換し、近さを比べます。

必ず見つかるわけではありません。第5回の結果と比較します。

比較前の予想

質問	予想する検索上の難しさ
SQLiteに保存するには？	資料と同じ語を含む
ページを閉じてもデータが残るのはなぜ？	言い換えた表現を含む
来年度の試験日は？	資料自体に答えがない

同じ資料とtop_kで、検索された本文を比べます。
予想と観察結果を別々に記録します。

ベクトルとは

数値を並べたものです。説明用の例：

```
a = [1.0, 0.0, 1.0]
b = [0.9, 0.1, 1.0]
c = [0.0, 1.0, 0.0]
```

実際のEmbeddingは、多数の次元を持ちます。
各数値を「価格」「色」のように直接解釈できるとは限りません。

説明用ベクトルの向き

実際のEmbeddingではなく、計算用の2次元の例です。

ベクトル	成分	aとの関係
a	[1, 0]	基準
b	[2, 0]	長さは違うが同じ向き
c	[0, 1]	直交する向き
d	[-1, 0]	反対向き

コサイン類似度では、長さよりも向きの関係を比べます。

生成用と検索用のモデル

用途	モデル	出力
回答を作る	gpt-6-luna	文章など
文章をベクトル化	text-embedding-3-small	数値の列

資料と質問には、同じEmbeddingモデルを使います。
別モデルの座標同士を、そのまま比較しません。

2つのタイミング

資料の登録・更新時	質問を受けたとき
断片を1つずつベクトル化	質問をベクトル化
ベクトルとモデル名を保存	保存済みの各ベクトルと比較
次の質問で再利用	上位の本文を取得して生成へ

資料のベクトルを毎回作り直す必要はありません。

登録時に残るもの

1. `all_chunks()` で資料の断片を取得する
2. 資料名・見出し・本文をつなぎ、`embed()` に渡す
3. ベクトルをJSON文字列にしてDBへ保存する
4. 使用したモデル名も同じ行へ保存する

原本の本文は、そのまま別のカラムに残ります。
質問のたびに、資料のEmbedding APIを呼び直す必要はありません。

質問時に行うこと

1. 保存済みベクトルと、現在のモード・モデル名を確認する
2. 質問を `embed()` に渡す
3. 質問と各資料のベクトルの類似度を計算する
4. 正のスコアの上位から本文を取得して、回答生成へ渡す

検索に使う数値と、生成に使う本文は役割が異なります。

Embedding API

```
response = client.embeddings.create(
    model=EMBEDDING_MODEL,
    input=text,
)
vector = response.data[0].embedding
```

第2回と同じクライアントを使いますが、呼ぶAPIが異なります。

この呼び出しでも外部へ文章を送信し、利用料金が発生します。

コサイン類似度

2つのベクトルの向きの近さを計算します。

内積を、それぞれの長さの積で割ります。

```
内積 = a[0]×b[0] + a[1]×b[1] + …
長さ = 各成分の2乗の合計の平方根
類似度 = 内積 ÷ (aの長さ × bの長さ)
```

同じ向きは1。ゼロベクトルは教材では0として扱います。

同じ向きの計算例

a = [1, 0]、b = [2, 0] の場合：

計算	結果
内積	$1 \times 2 + 0 \times 0 = 2$
aの長さ	$\sqrt{(1^2 + 0^2)} = 1$
bの長さ	$\sqrt{(2^2 + 0^2)} = 2$
コサイン類似度	$2 \div (1 \times 2) = 1$

長さが2倍でも、同じ向きなら類似度は1です。

向きが異なる計算例

a = [1, 0] を基準にします。

比較相手	内積	長さの積	類似度
[0, 1]	0	1	0
[-1, 0]	-1	1	-1

ゼロベクトルは長さが0なので、そのままでは割り算できません。
教材は、どちらかがゼロベクトルなら0を返します。

forで内積を計算

```
for i in range(len(a)):
    dot += a[i] * b[i]
    a_size += a[i] * a[i]
    b_size += b[i] * b[i]
```

`i` はリストの位置です。同じ位置の成分同士を掛けます。
教材では専用の数値計算ライブラリを使わず、処理を追えるようにします。

ループの途中の値

a = [1, 2]、b = [2, 1] で計算します。

段階	dot	a_size	b_size
初期値	0	0	0
i = 0 の後	2	1	4
i = 1 の後	4	5	5

類似度は $4 \div (\sqrt{5} \times \sqrt{5}) = 0.8$ です。

`a_size` と `b_size` は、この段階では長さの2乗です。

実習1：計算の穴埋め

`embeddings.py` の `cosine_similarity()` を完成させます。
コメントを外し、仮の `raise` を削除します。

ターミナルで `python` を起動して試します。

```
from embeddings import cosine_similarity
print(cosine_similarity([1, 0], [1, 0]))  # 1
print(cosine_similarity([1, 0], [0, 1]))  # 0
```

計算結果の確認

入力	期待する結果
[1, 0] と [2, 0]	1
[1, 0] と [0, 1]	0
[1, 2] と [2, 1]	約0.8
[0, 0] と [1, 0]	教材の取り決めで0
[1] と [1, 0]	長さの不一致でValueError

小数の計算では、表示の末尾に丸めの差が出ることがあります。

デモ用ベクトルの注意

デモは、指定した単語の出現回数を数値にしています。

- 外部のEmbeddingモデルを使わない
- 意味や言い換えを理解しない
- ベクトルの保存・比較の流れを観察するために使う
- 本物の意味検索の品質評価には使わない

デモとAPIモードの結果を混ぜて採点しません。

デモ用ベクトルの中身

デモの数値は、次の語の出現回数をこの順番で並べたものです。

`sqlite, 保存, fetch, 提出, api, キー, css, sql`

```
「保存 保存」      [0, 2, 0, 0, 0, 0, 0, 0]
「データが残る」    [0, 0, 0, 0, 0, 0, 0, 0]
```

意味が近くても、指定語がなければゼロベクトルになります。
この結果を、本物のEmbeddingモデルの性能と解釈しません。

実習2：検索の準備

```
cd session06/exercise
python init_db.py
python build_index.py
python main.py
```

APIモードは `.env` または環境変数を先に設定します。
`build_index.py` は、APIモードなら各断片を外部へ送ります。
初回は資料数を確認し、講師の案内に従って実行します。

DBに保存するもの

項目	保存する理由
本文	LLMへ渡すのは元の文章
embedding	次の検索で再利用する
embedding_model	質問と同じモデルか確かめる
ファイル名・見出し	根拠を確認する

ベクトルだけをLLMへ渡しても、この教材の回答は作れません。

実習3：同じ質問で比較

APIモードで次の質問を比較します。

1. 「SQLiteに保存するには？」
2. 「ページを閉じてもデータが残るのはなぜ？」
3. 「第7回の提出物は？」
4. 「来年度の試験日は？」

キーワード検索と意味検索で、選ばれた資料を記録します。
比較する上位件数と資料の版をそろえます。

比較表に残す情報

同じ質問について、方式ごとに1行ずつ記録します。

記録する項目	見る内容
検索方式と入力	キーワードの検索語、意味検索の質問
資料とtop_k	原本の版と上位件数をそろえる
取得した本文	正解の根拠が含まれるか
回答	根拠に沿うか、根拠不足で保留できるか

異なる検索方式のスコアの数値だけで、優劣を決めません。

スコアの読み方

類似度は、回答が正しい確率ではありません。
また、キーワード検索の点数と同じ尺度でもありません。

- 高いスコアでも、質問の答えが含まれない場合がある
- 固有名詞や記号はキーワード検索が役立つ場合もある
- 件数やしきい値は、評価用質問で検討する

この教材は正のスコアから上位件数を選ぶ単純な実装です。

資料を更新した場合

```
python init_db.py
python build_index.py
```

再登録は、古いベクトルを消します。
モデルやデモ/APIモードを変えた場合も、再作成が必要です。

本文と古いベクトルの組み合わせで検索しないための仕組みです。

ベクトルの不一致を直す手順

状態	対応
原本の本文を変更した	init_db.py、その後build_index.py
デモからAPIに切り替えた	新しい設定でbuild_index.py
Embeddingモデルを変更した	新しいモデルでbuild_index.py

APIモードでは資料のベクトル化にも外部通信が発生します。
エラーを隠すためにモデル名だけを書き換えず、ベクトルを再作成します。

費用と規模

質問1回の意味検索では、通常、質問のEmbeddingと回答生成を呼びます。
資料のEmbedding作成にも別に費用がかかります。

この教材は少量の資料をPythonのループで全件比較します。
大規模な検索基盤の運用は、発展課題とします。
まずは検索精度と資料の整え方を学びます。

確認問題

1. 資料と質問に異なるEmbeddingモデルを使ってよい？
2. 類似度0.8は正答率80%？
3. 意味検索なら、根拠不足への対処は不要？
4. 資料を更新した後、何を作り直す？

提出物と次回

提出：`embeddings.py` と `worksheet.md` のGitHub URL。

2つの検索方式について、成功例と失敗例を1つずつ記録します。
APIを使えなかった場合は、その旨を明記し、講師の実演を観察します。

次回は、文章の検索に加えて、課題DBの値を取得します。